



**CSE3433 SOFTWARE ARCHITECTURE
K2**

Sem II 2025/2026

**Program Sarjana Muda Sains Komputer (Kejuruteraan
perisian) Dengan Kepujian**

GROUP 15

Project Proposal

NAMA PENSYARAH: DR MOHAMAD NOR

	NAMA	NO MATIRK
1	MUHAMMAD RADILAN BIN YAAKUB	S75736
2	MUHAMMAD ADAM DANISH BIN MOHAMAD FAIZ	S75851

Table of Contents

1. Architecture Design.....	2
1.1 System Overview.....	2
1.2 Functional Requirements.....	2
1.3 Non-functional Requirements.....	3
1.4 Architecture Style Explanation.....	3
1.5 Service or Component Identification.....	4
1.6 Architecture Diagrams.....	5
1.6.1 System Context Diagram.....	5
1.6.2 Component / Service Architecture Diagram.....	6
1.6.3 Interaction Diagram.....	7
1.6.4 Deployment Diagram.....	8
References.....	9

1. Architecture Design

1.1 System Overview

Event-Driven Job Recruitment & Application Processing System is the system prototype that have been create for modernize and automated the recruitment process with more efficiently and systematically. This system allows two types of user which is employers and job seeker have interaction in one platform. Employers can advertise vacancies while job seeker can apply work online. Each interaction in this system such as submitting applications, updating status, and scheduling interviews will trigger certain events that will be processed automatically by other components.

Furthermore, this system operates asynchronously and does not depend directly on each other. Therefore, it is also increasing the flexibility and scalability of the system. For example, when a user submits a job application, the system will generate an “Application Submitted” event which will then trigger other processes such as data storage, initial assessment, and sending notifications to users. This approach will allow the system to provide real-time responses and reduce delays in data processing. As a result, this system aims to provide a more responsive, modular, and suitable solution for the needs of modern systems in recruitment management.

1.2 Functional Requirements

To get the job done, the system focuses on a few non-negotiable features:

- **Job Lifecycle Management:** Hiring managers need to be able to spin up new job requirements and have those requirements automatically blast out to internal and external boards via event triggers.
- **Smooth Application Flow:** Candidates need a clean portal where they can drop their details and resumes. The system must acknowledge receipt instantly, even if the backend is still "thinking".
- **Intelligence on the Fly:** We need a background service that automatically rips data out of PDFs (parsing) and matches candidate skills against the job's "must-haves" (scoring).
- **Real-time Status Loop:** Every time a candidate moves from "Applied" to "Shortlisted" or "Rejected," the system needs to fire off a notification so no one is left in the dark.

1.3 Non-functional Requirements

This is where the "engineering" happens to ensure the system doesn't break under pressure:

- **Elastic Scalability:** If a viral job post brings in 10,000 applications in an hour, the message broker (like Kafka or RabbitMQ) acts as a buffer so our database doesn't catch fire.
- **High Availability:** The candidate portal should almost never be "down". By decoupling the services, even if the "Resume Scorer" is undergoing an update, candidates can still submit their applications.
- **Bulletproof Persistence:** No application should ever be "lost". Using an event-driven approach means if a service crashes, the event stays in the queue until it's successfully processed.
- **Responsiveness:** The goal is a sub-2-second response time for any front-end action, offloading all the "heavy" work to background consumers.

1.4 Architecture Style Explanation

We chose **Event-Driven Architecture (EDA)** because, frankly, the old way of doing things is too rigid for modern hiring. In a traditional setup, if the email server is slow, the whole application process feels slow for the candidate. In this architecture, every actions or change in this system will create one event that will be sent to another komponen through intermediary which is event broker. The components in the system do not communicate directly, but instead respond to the events received. It is also makes the system loosely coupled and more flexible to develop and expand. This approach is very suitable for recruitment systems that involve various processes such as submitting applications, evaluating candidates, and sending notifications that occur continuously.

Furthermore, EDA support asynchronous processing where every event can be process without waiting for immediate response from another component. Therefore, the system can operate more efficiently and reduce the risk of delay especially when numbers of user increase. For example, when event like "Application Submitted" is generate, the system will simultaneously save the data of application and then the system will be send notification to the user. After that, the system will start the evaluation process without disturbing main flow of the system. For the conclusion, the use of Event-Driven Architecture not only improves the performance and scalability of the system, but it is also ensures that the system is easier to maintain and suitable for the needs of modern applications.

1.5 Service or Component Identification

1. API Gateway

Acts as the main entry gate for all user request which will route the request to the appropriate service as well as manage authentication and communication control.

2. Event Bus (Kafka or RabbitMQ)

Function as a communication layer which enable interaction between service asynchronous through publishing concept and event subscription.

3. User Management Service

Manage all function related to user including application, log in, identity verification, and profile management for job seeker and employers.

4. Job Service

Handle all operations related to the job vacancies such as creating, updating, deleting, and displaying job postings in the system.

5. Interview Scheduling Service

Responsible to manage interview scheduling including date and time, and also coordinating between employers and candidates.

6. Application Processing Service

The engine for handling incoming applications, resume storage, and the state-machine logic for candidate progression.

7. Notification Service

Send notifications to the user in real time based on event in the system such as update application status and interview schedule.

1.6 Architecture Diagrams

1.6.1 System Context Diagram

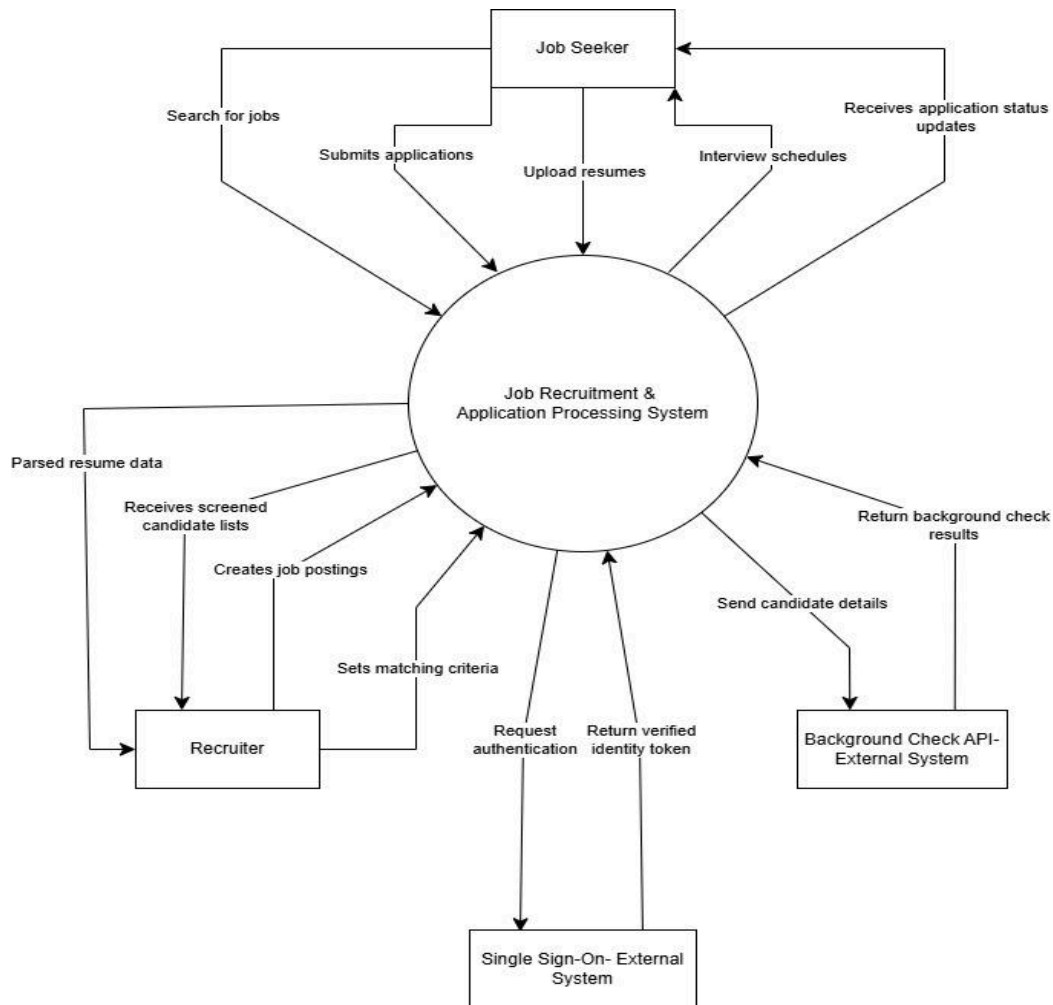


Diagram 1 : System Context Diagram

The system illustrates a **Job Recruitment & Application Processing System** as a main central which is connects multiple user and external system. **Job seeker** interacts with the the system for searching a job, submitting applications, and uploading resumes . It is also can receive interview schedule and update application status. On other side, **recruiter** use the system to create job postings, sets candidate criteria and also receive list of candidates which is have been screened along with the processed resume data. This system also integrate with the **Single Sign-On External** for the identity verification where system will send an authentication request and receive a verified identity token to ensure access security. In addition, the system also communicates with the **Background Check API External** system by sending candidate information and receiving back the results of the check.

1.6.2 Component / Service Architecture Diagram

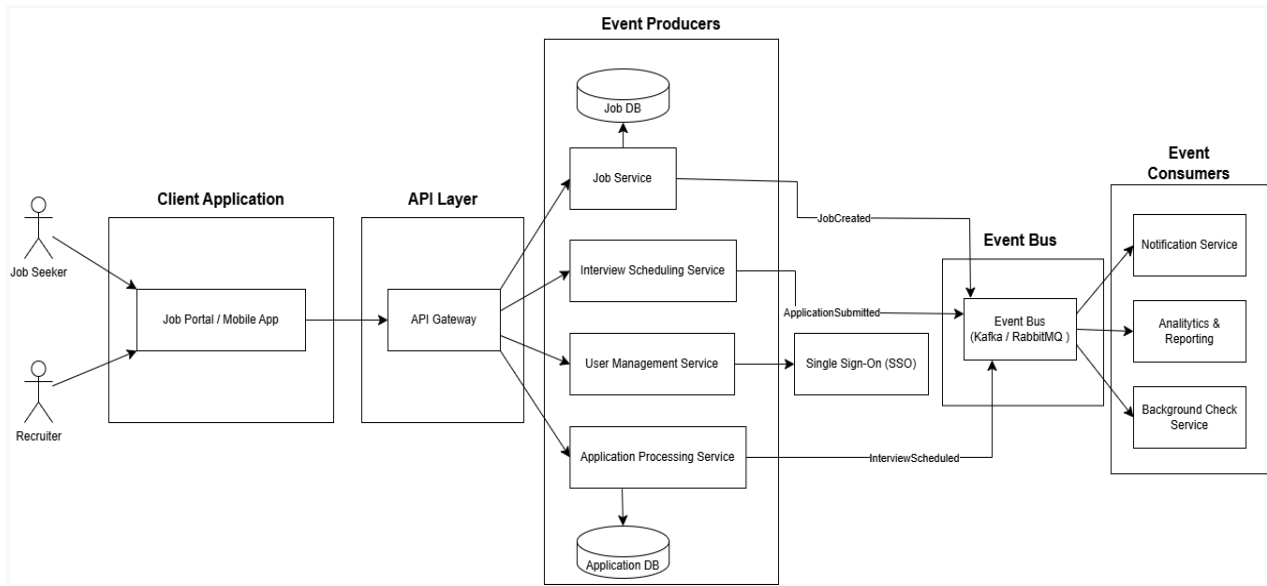


Diagram 2 : Component/Service Architecture Diagram

The Component / Service Architecture diagram illustrates an event-driven architecture for the Job Recruitment and Application Processing System. User such as Job Seeker And Recruiter interact through the client application which is Job Portal or Mobile App. It will be send a request to the API Gateway in the API Layer. After that, gateway distributes requests to several services such as Job Service (managing job data), Interview Scheduling Service (scheduling interviews), User Management Service (user management and SSO), and Application Processing Service (processing applications). Each service can store data in its own database and also produce events such as “JobCreated”, “ApplicationSubmitted”, and “InterviewScheduled” which are sent to the Event Bus (Kafka or RabbitMQ). The Event Bus then delivers these events to event consumers such as Notification Service, Analytics & Reporting, and Background Check Service for further action asynchronously, making the system more flexible, scalable, and loosely coupled.

1.6.3 Interaction Diagram

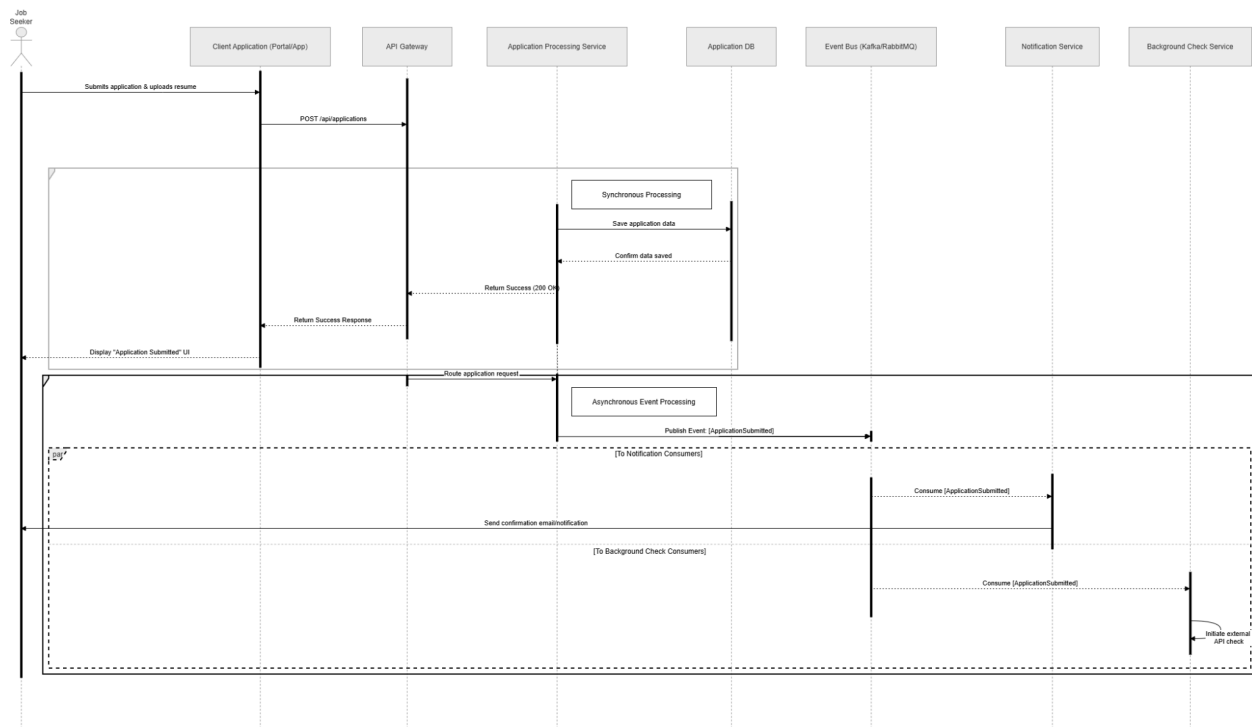


Diagram 3 : Interaction Diagram

The steps involved in a job seeker submitting an application are depicted in the interaction diagram. The process begins with the **Client Application** sending the application data to the **API Gateway**, which then forwards the request to the **Application Processing Service**. An **Application Submitted** event is instantly published to the Event Bus via this service. After that, the Event Bus initiates two simultaneous processes: the **Notification Service** gives the user an immediate confirmation, and the **Application Processing Service**, operating as a background consumer, does a preliminary assessment. Following evaluation, the consumer publishes an **ApplicationReviewed** event, which causes the **Notification Service** to provide a final status update. This flow illustrates the Event-Driven Architecture's asynchronous and decoupled design, which guarantees the system's responsiveness even when processing demanding activities.

1.6.4 Deployment Diagram

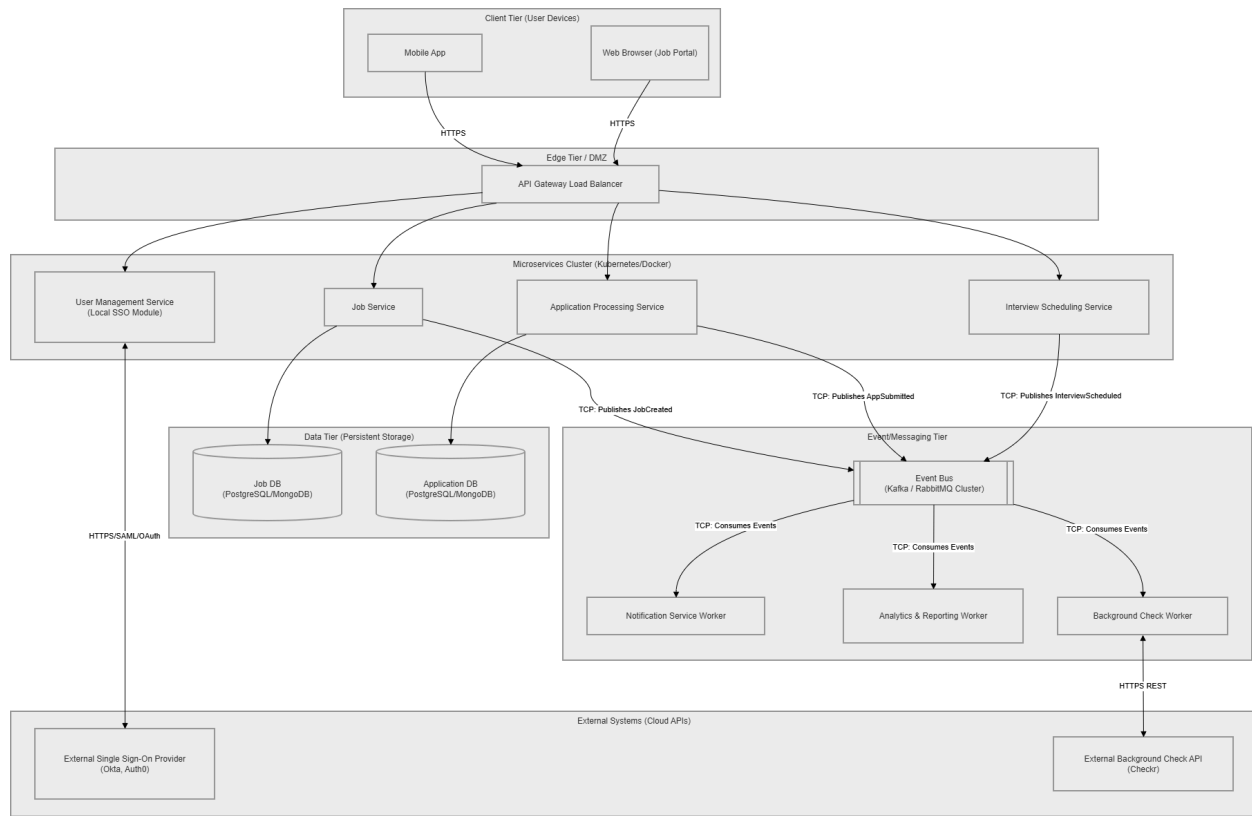


Diagram 4 : Deployment Diagram

The Event-Driven Job Requirement & Application Processing System's deployment environment and physical infrastructure are depicted in the deployment diagram. Request routing and traffic management are handled by an API gateway, which controls client access through web or mobile applications. A collection of decoupled **Microservices** like User Service, Job Service, and Application Processing Service that are deployed in containers like Docker for high availability and elastic scalability provide the system's essential functionality. The **Event Bus** (Kafka or RabbitMQ), which is necessary for controlling the asynchronous communication between all services, is hosted on a dedicated server node. The Data Layer ensures high availability and unbreakable durability by maintaining persistent data across one or more databases.

References

1. *What is event-driven architecture? A complete guide.* (2025, September 11). Technology & Software Development Blog | Future Processing.
<https://www.future-processing.com/blog/event-driven-architecture-guide/>
2. GeeksforGeeks. (2024, January 31). *EventDriven Architecture System Design.* GeeksforGeeks.
<https://www.geeksforgeeks.org/system-design/event-driven-architecture-system-design>
3. Richardson, C. (2017). *Microservices.io.* Microservices.io; Chris Richardson.
<https://microservices.io/patterns/data/event-driven-architecture.html>
4. Navarro, J., & Parrend, P. (2018). *Clustering analysis classes to identify software component*, 76.
<https://www.sciencedirect.com/science/article/abs/pii/S095058491400130X>
5. sallam, mohammed. (2025, August 26). *Fundamentals of software architecture Chapter 14: Event-Driven Architecture Style.* Medium.
<https://medium.com/@mohamedsallam953/fundamentals-of-software-architecture-chapter-14-event-driven-architecture-style-ffea4ecbbce1>